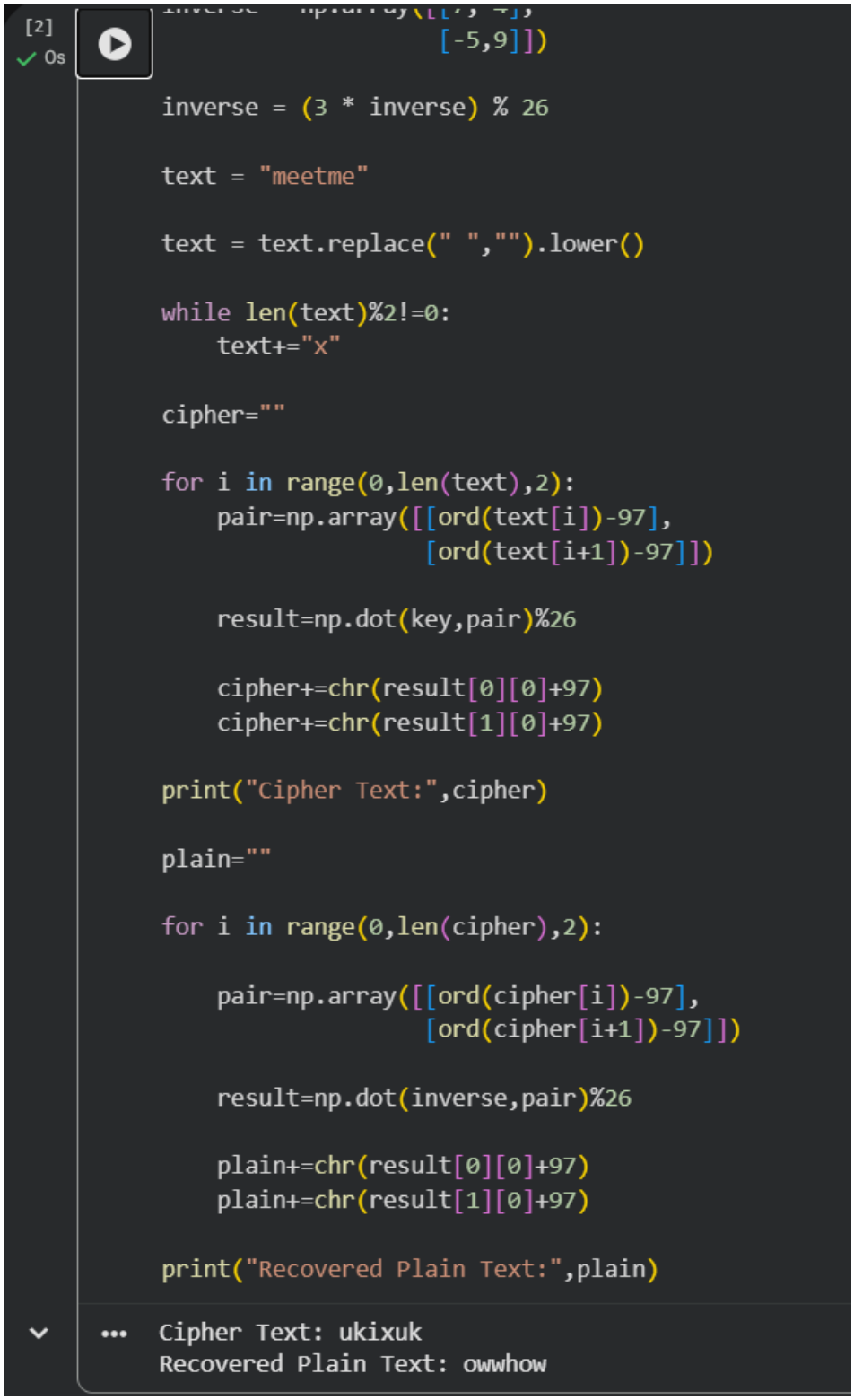
11. Playfair Cipher Key Count

[1]
✓ 0s

```python
import math

# Total possible Playfair keys = 25!
keys = math.factorial(25)

print("Total possible Playfair keys (25!):")
print(keys)

bits = math.log2(keys)
print("\nApproximate power of 2:")
print("≈ 2^{:.2f}".format(bits))

# Effective unique keys
effective = keys // 2

print("\nApproximate effectively unique keys:")
print(effective)
```

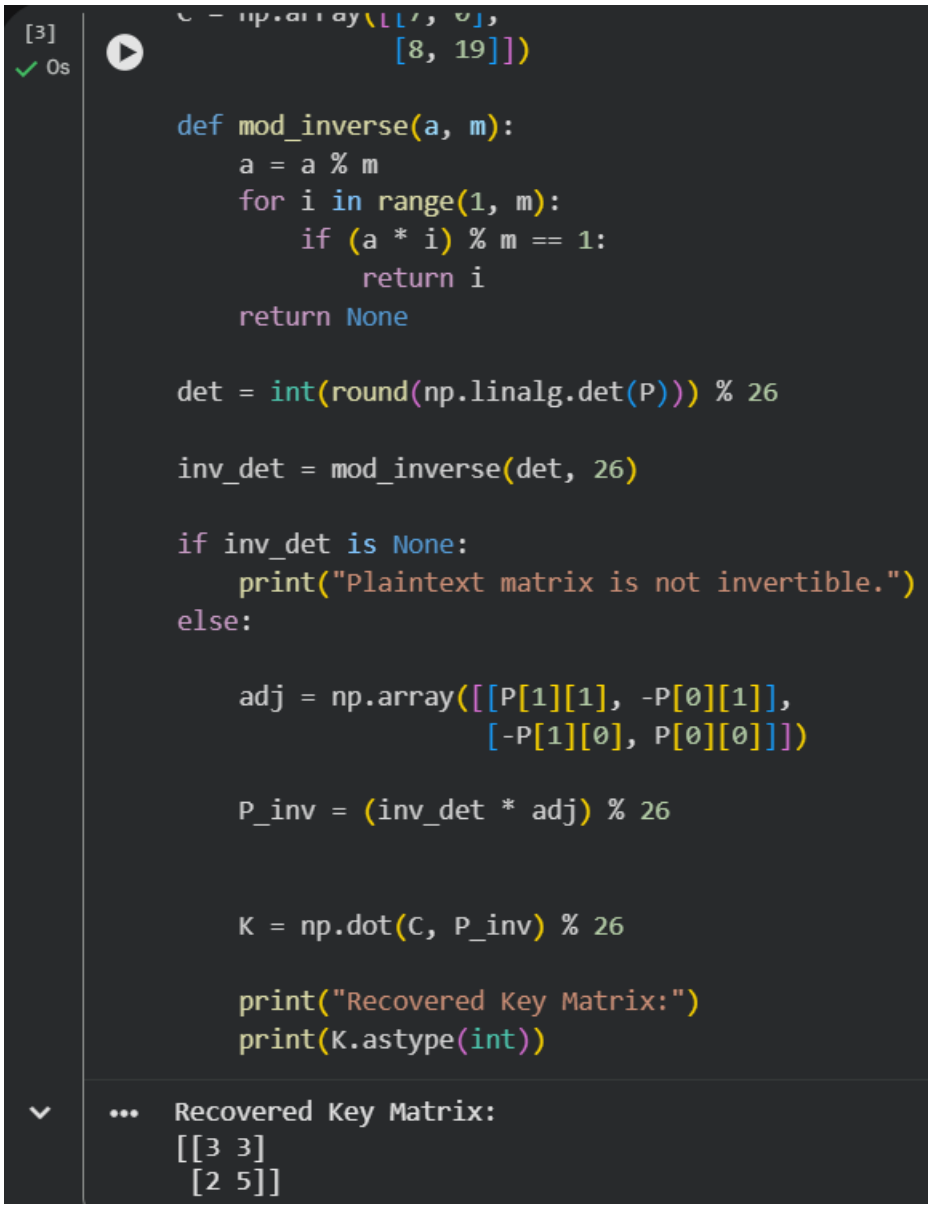
```
Total possible Playfair keys (25!):
15511210043330985984000000

Approximate power of 2:
≈ 2^83.68

Approximate effectively unique keys:
7755605021665492992000000
```

12. Hill Cipher Encryption & Decryption



```
[2]
✓ 0s
                    [-5,9]])

inverse = (3 * inverse) % 26

text = "meetme"

text = text.replace(" ","").lower()

while len(text)%2!=0:
    text+="x"

cipher=""

for i in range(0,len(text),2):
    pair=np.array([[ord(text[i])-97],
                   [ord(text[i+1])-97]])

    result=np.dot(key,pair)%26

    cipher+=chr(result[0][0]+97)
    cipher+=chr(result[1][0]+97)

print("Cipher Text:",cipher)

plain=""

for i in range(0,len(cipher),2):

    pair=np.array([[ord(cipher[i])-97],
                   [ord(cipher[i+1])-97]])

    result=np.dot(inverse,pair)%26

    plain+=chr(result[0][0]+97)
    plain+=chr(result[1][0]+97)

print("Recovered Plain Text:",plain)

...  Cipher Text: ukixuk
     Recovered Plain Text: owwhow
```

13. Hill Cipher Known Plaintext Attack



```
[3]
✓ 0s
                [8, 19]])

def mod_inverse(a, m):
    a = a % m
    for i in range(1, m):
        if (a * i) % m == 1:
            return i
    return None

det = int(round(np.linalg.det(P))) % 26

inv_det = mod_inverse(det, 26)

if inv_det is None:
    print("Plaintext matrix is not invertible.")
else:

    adj = np.array([[P[1][1], -P[0][1]],
                    [-P[1][0], P[0][0]]])

    P_inv = (inv_det * adj) % 26


    K = np.dot(C, P_inv) % 26

    print("Recovered Key Matrix:")
    print(K.astype(int))

...  Recovered Key Matrix:
[[3 3]
 [2 5]]
```

14(a). One-Time Pad (Vigenère) Encryption

```
[4]
✓ 0s
```

```python
text = "sendmoremoney".replace(" ","").lower()

key = [9,0,1,7,23,15,21,14,11,11,2,8,9]

cipher=""

for i in range(len(text)):
    p = ord(text[i])-97
    c = (p + key[i]) % 26
    cipher += chr(c+97)

print("Plaintext :",text)
print("Ciphertext:",cipher)
```

```
Plaintext : sendmoremoney
Ciphertext: beokjdmsxzpmh
```

14(b). Find Key for "cashnotneeded"

```
[5]
✓ 0s
```

```python
cipher = "beoljlmshwjmg"

plain = "cashnotneeded".replace(" ","")

key=[]

for i in range(len(cipher)):
    c = ord(cipher[i])-97
    p = ord(plain[i])-97
    k = (c-p)%26
    key.append(k)

print("Required Key Stream:")
print(key)
```

```
Required Key Stream:
[25, 4, 22, 4, 22, 23, 19, 5, 3, 18, 6, 8, 3]
```

15. Letter Frequency Attack on Additive Cipher

[6]
✓ 15s

```
cipher = input("Enter Cipher Text: ").lower()

print("\nTop 10 Possible Plaintexts\n")

for key in range(10):

    plain=""

    for ch in cipher:
        if ch.isalpha():
            p=(ord(ch)-97-key)%26
            plain+=chr(p+97)
        else:
            plain+=ch

    print("Key",key,":",plain)
```

```
Enter Cipher Text: khoor

Top 10 Possible Plaintexts

Key 0 : khoor
Key 1 : jgnnq
Key 2 : ifmmp
Key 3 : hello
Key 4 : gdkkn
Key 5 : fcjjm
Key 6 : ebiil
Key 7 : dahhk
Key 8 : czggj
Key 9 : byffi
```